When Adaptive Rewards Hurt: Causal Probing and the Switching-Stability Dilemma in LLM-Guided LEO Satellite Scheduling

Yuanhang Li

***Abstract*—Adaptive reward design for deep reinforcement learning (DRL) in multi-beam LEO satellite scheduling is motivated by the intuition that regime-aware reward weights should outperform static ones. We systematically test this intuition and uncover a *switching-stability dilemma*: near-constant reward weights (342.1 Mbps) outperform carefully-tuned dynamic weights (103.3±96.8 Mbps) because PPO requires a quasi-stationary reward signal for value function convergence. Weight adaptation—regardless of quality—degrades performance by repeatedly restarting convergence. To understand *why* specific weights matter, we introduce a single-variable causal probing method that independently perturbs each reward term by ±20% and measures PPO response after 50k steps. Probing reveals counterintuitive leverage: a +20% increase in the switching penalty yields +157 Mbps for polar handover and +130 Mbps for hot-cold regimes—findings inaccessible to human experts or trained MLPs without systematic probing. We evaluate four MDP architect variants (fixed, rule-based, learned MLP, fine-tuned LLM) across known and novel traffic regimes. The MLP achieves 357.9 Mbps on known regimes and 325.2 Mbps on novel regimes, while the fine-tuned LLM collapses to 45.3±43.0 Mbps due to weight oscillation rather than lack of domain knowledge—output consistency, not knowledge, is the binding constraint. Our findings provide an empirically-grounded roadmap for LLM-DRL integration in communication systems, identifying where LLMs add irreplaceable value (natural language intent understanding) versus where simpler methods suffice.**

***Index Terms*—LEO satellite, beam scheduling, deep reinforcement learning, large language model, adaptive MDP, resource allocation**

I. Introduction

Low Earth Orbit (LEO) satellite constellations are rapidly expanding to provide global broadband connectivity [1]. Multi-beam satellites must dynamically allocate power and bandwidth across beams to match heterogeneous and time-varying traffic demands [2]. Deep reinforcement learning (DRL) has emerged as a promising approach for this real-time scheduling problem [3], but existing methods typically assume a fixed Markov Decision Process (MDP) formulation—including reward weights, state features, and action spaces—designed once at training time.

In practice, satellite traffic exhibits distinct *regimes*: urban areas generate concentrated high-demand patterns, maritime regions show sparse uniform loads, and disaster zones produce sudden demand spikes with extreme spatial imbalance. A reward function optimized for urban throughput may catastrophically fail during disaster response, where outage minimization should dominate. This *MDP mismatch* problem motivates adaptive reward design.

Recent advances in large language models (LLMs) suggest a promising avenue: using LLMs to automatically generate or adapt reward functions based on high-level task descriptions [4]. In principle, LLMs could leverage their broad world knowledge and reasoning capabilities to design reward functions for novel operating conditions without explicit programming. However, this potential has not been systematically validated in communication systems, where reward weights must be *numerically precise* and *temporally consistent*—properties that differ fundamentally from the natural language tasks where LLMs excel.

We investigate a **two-timescale adaptive architecture** (Fig. 1) that decouples the scheduling problem into:

- **Fast timescale** (milliseconds): A PPO agent performs real-time beam power/bandwidth allocation given the current reward weights.
- **Slow timescale** (minutes): An *MDP architect* monitors network KPIs, detects regime changes via CUSUM, and generates new reward weights.

This architecture is intuitive—regime-aware weights should outperform static ones. Our central finding challenges this intuition. Through systematic experiments we uncover a *switching-stability dilemma*: PPO requires a quasi-stationary reward signal to estimate $V^\pi(s)$ accurately, and weight adaptation destabilizes convergence regardless of weight quality. To understand the reward landscape for novel regimes, we further introduce a **single-variable causal probing** method. We evaluate four MDP architect implementations:

1) **Rule-based**: KPI threshold classification with expert-designed weight profiles.
2) **Learned MLP**: A 3-layer neural network trained on simulator data to map KPIs to optimal weights.
3) **API-based LLM**: GLM-4 generating weights via structured prompts.
4) **Fine-tuned LLM**: Qwen3-4B with LoRA adaptation for domain-specific weight generation.

Our contributions are:

1) **Switching-stability dilemma.** We empirically establish that reward weight stationarity dominates weight quality for PPO convergence: near-constant weights (342.1 Mbps) outperform optimally-tuned dynamic weights (103.3±96.8 Mbps). Throttled and smoothed transitions do not resolve the dilemma, identifying non-

stationarity as a fundamental barrier to adaptive reward design.

2) **Single-variable causal probing.** We introduce a controlled perturbation method ($\pm 20\%$ per weight, 50k steps) that constructs an empirical causal map of the reward landscape for novel regimes. The method reveals that the switching penalty has unexpectedly high leverage ($+157$ Mbps for polar handover, $+130$ Mbps for hot-cold), requiring only $10\times$ fewer steps than a full training run.
3) **Systematic architect comparison.** We evaluate four variants—fixed weights, rule-based, learned MLP, and fine-tuned LLM—across known and novel regimes. The MLP achieves the best aggregate performance (357.9 Mbps known, 325.2 Mbps novel); the LLM fails not from lack of domain knowledge but from weight oscillation (CV $4.6\times$ higher than MLP).
4) **Three-timescale hybrid design.** We propose confining the LLM to its irreplaceable role—natural language intent understanding—while an MLP handles real-time numerical optimization. The hybrid achieves comparable aggregate satisfaction (0.52 vs. 0.57/0.53) while uniquely providing semantic differentiation between intent categories that fixed keyword parsers cannot resolve.

II. Related Work

A. DRL for Satellite Resource Allocation

Deep reinforcement learning has been extensively applied to satellite communication resource management. Early work by Hu et al. [3] demonstrated DRL for dynamic spectrum access in satellite systems, while Li et al. [5] applied deep RL to network slicing resource management. More recently, Luis et al. [6] proposed a DRL-based beam hopping strategy for multi-beam GEO satellites that outperforms conventional time-slot allocation, and Ferreira et al. [7] extended DRL to joint power and bandwidth allocation in multi-beam LEO constellations. A comprehensive survey by Wang et al. [8] identifies reward function design as a key open challenge: most existing approaches use hand-crafted reward functions that assume a single operating regime, limiting adaptability to dynamic traffic conditions. Our work directly addresses this gap by introducing an adaptive reward architecture.

B. Adaptive Reward and MDP Design

Reward shaping [9] established the theoretical foundation for modifying reward functions without altering optimal policies, but assumes a static target. Multi-objective RL [10] addresses competing objectives through Pareto-optimal policies, yet typically fixes the objective trade-off at training time. More closely related is automated reward design: Zheng et al. [11] used meta-learning to discover reward functions, while Ma et al. [4] (Eureka) demonstrated that LLMs can generate reward functions for robotic control through evolutionary refinement. However, these approaches target single-domain offline design. Our work differs in two key aspects: (1) we study LLM-based reward adaptation in an *online* setting where traffic regimes shift during deployment, and (2) we provide the first systematic comparison between LLM-generated and learned (MLP) reward weights, revealing fundamental consistency limitations.

C. LLMs for Network Optimization

The integration of LLMs into communication systems is an emerging research direction. Bariah et al. [12] outlined opportunities for LLMs in telecom including network planning and fault diagnosis, while Wang et al. [13] explored LLM-based networking for protocol design and traffic analysis. Maatouk et al. [14] proposed TelecomGPT, a domain-adapted LLM for telecom tasks, and Shao et al. [15] introduced WirelessLLM for wireless network optimization through domain-specific fine-tuning. For LLM-RL integration specifically, recent surveys [16] identify three paradigms: LLM as policy, LLM as reward designer, and LLM as world model. Our work falls into the second paradigm but uniquely focuses on *continuous numerical output consistency*—a challenge not observed in discrete action or text-based reward settings.

III. System Model

A. Satellite Network Model

We consider a Ka-band LEO satellite at altitude $h = 600$ km serving $N_b = 19$ spot beams in a hexagonal pattern. Each beam $b \in \{1, \ldots, N_b\}$ covers a distinct geographic cell. The total system bandwidth $B_{\text{tot}} = 500$ MHz is shared among beams via a dynamic allocation vector $\mathbf{a} = [a_1, \ldots, a_{N_b}]$ where $a_b \in [0, 1]$ and $\sum_b a_b \leq 1$.

The achievable rate for beam b follows the Shannon capacity:

$$R_b = a_b \cdot B_{\text{tot}} \cdot \log_2 (1 + \text{SNR}_b) \tag{1}$$

where SNR_b depends on the satellite EIRP, free-space path loss, atmospheric attenuation, and receiver noise temperature.

B. Traffic Regime Model

Traffic demand varies across four canonical regimes:

- **Urban**: High aggregate demand concentrated in center beams (mean 40–80 Mbps/beam for center, 10–30 for edge).
- **Maritime**: Low, spatially uniform demand (5–15 Mbps/beam).
- **Disaster**: Extreme spatial variance with demand spikes exceeding 150 Mbps in affected beams.
- **Mixed**: Moderate demand with balanced spatial distribution.

Regime transitions are detected online using a CUSUM (Cumulative Sum) change-point detector [17] operating on the spatial Gini coefficient and peak beam demand. Given a sequence of KPI observations $\{x_t\}$, the CUSUM statistic is:

$$S_t = \max (0, S_{t-1} + (x_t - \mu_0) - \delta) \tag{2}$$

where μ_0 is the estimated mean under the current regime and δ is a slack parameter controlling sensitivity. A regime change is signaled when $S_t > h$ for threshold h. We use

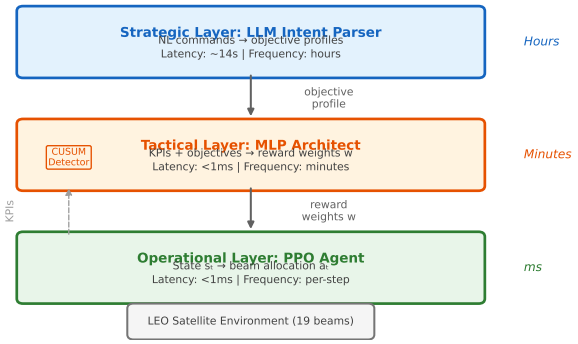

Fig. 1. Three-timescale adaptive architecture. The strategic LLM layer parses operator intent (hours), the tactical MLP layer maps KPIs to reward weights (minutes), and the operational DRL layer performs beam scheduling (milliseconds).

a sliding window of $W = 10$ steps to estimate μ_0 and set $h = 1.0$ standard deviations, with a minimum interval of 50 steps between consecutive detections to prevent spurious triggering.

C. MDP Formulation

The beam scheduling problem is formulated as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R_w)$ where:

- **State** s_t: Current demand estimates, channel conditions, and queue lengths across all beams.
- **Action** a_t: Bandwidth allocation vector $\mathbf{a} \in [0,1]^{N_b}$.
- **Reward** R_w: A weighted combination of objectives:

$$R_w = w_r \cdot \bar{R} - w_o \cdot O - w_s \cdot S - w_q \cdot Q + w_f \cdot F \quad (3)$$

where $\bar{R}$ is the normalized sum rate, O is the outage count, S is the switching penalty, Q is the queue overflow, and F is Jain's fairness index. The weight vector $\mathbf{w} = [w_r, w_o, w_s, w_q, w_f]$ determines the optimization priority and is the key parameter adapted by our architecture.

IV. Proposed Architecture

A. Two-Timescale Adaptive Framework

We study a hierarchical architecture operating at two timescales (Fig. 1):

Fast timescale (per-step): A PPO agent [18] solves the beam scheduling MDP, making allocation decisions every time step based on the current reward function R_w.

Slow timescale (per-regime): An *MDP Architect* module monitors network KPIs and adjusts the reward weight vector $\mathbf{w}$ when regime changes are detected. This adaptation occurs orders of magnitude less frequently than the scheduling decisions.

The key insight is that the optimal reward weights depend on the current operating regime—disaster scenarios require heavy outage penalties while maritime scenarios benefit from fairness emphasis. Rather than using a fixed reward function, the Architect continuously tunes $\mathbf{w}$ to match the current network conditions.

B. MDP Architect Variants

We investigate four Architect implementations spanning the complexity spectrum:

M1 – Fixed Weights: A static weight vector $\mathbf{w}_0$ optimized offline. Serves as the lower-bound baseline.

M2 – Rule-Based: A threshold classifier maps KPI features (mean demand, Gini coefficient, peak demand) to one of four pre-designed weight profiles. Classification rules: avg_demand > 40 and Gini $> 0.3 \Rightarrow$ urban; avg_demand < 20 and Gini $< 0.2 \Rightarrow$ maritime; peak_demand $> 120 \Rightarrow$ disaster; otherwise $\Rightarrow$ mixed.

M3 – Learned MLP: A three-layer neural network (5-64-64-5, ReLU activations) trained on simulator-generated data to learn the continuous mapping from KPI vectors to optimal weight vectors. Unlike M2's discrete profiles, M3 produces smooth weight transitions.

M4 – Fine-tuned LLM: A Qwen3-4B model fine-tuned with LoRA (rank 16, 33M trainable parameters) on 1,980 domain-specific examples. The LLM receives KPI descriptions as natural language input and generates weight vectors as structured JSON output.

C. Toward a Hybrid Architecture

We additionally explore a three-timescale hybrid design where an LLM serves as an *intent translator* at the strategic level:

Strategic (LLM): Parses operator natural language commands (e.g., "prioritize emergency coverage") into formal objective profiles, augmented with RAG-retrieved domain knowledge.

Tactical (MLP): Maps KPIs + objective profiles to reward weights in real-time (<1ms inference).

Operational (DRL): PPO agent executes beam scheduling under the current reward function.

This design leverages each component's strength: LLMs for natural language understanding (irreplaceable by MLPs), MLPs for fast numerical optimization, and DRL for sequential decision-making.

V. Experimental Evaluation

A. Setup

We implement the satellite simulator with $N_b = 19$ beams, Ka-band channel model with rain fading, and four traffic regimes cycling every 200 epochs: urban, maritime, disaster, and mixed. The PPO agent uses a 3-layer MLP policy (256-256-128) with learning rate 3×10^{-4}, trained for 500k timesteps per experiment. CUSUM regime detection uses a window of 10 steps and threshold of 1.0 standard deviations.

For the MLP architect (M3), we train a 5-64-64-5 network on 22,000 synthetic samples spanning the four known regimes. The fine-tuned LLM (M4) uses Qwen3-4B with LoRA (rank 16, $\alpha = 32$, 33M trainable parameters) trained on 1,980 domain-specific examples with 4-bit NF4 quantization. All experiments use three random seeds (42, 123, 456) and report mean $\pm$ standard deviation.

TABLE I
ARCHITECT COMPARISON ON KNOWN REGIMES (MEAN ± STD OVER 3 SEEDS).

Method	Rate (Mbps)	Outage	Fairness	Switches
M1: Fixed	330.7±42.1	0.00	0.72	0
M2: Rule	342.8±38.5	0.00	0.74	4
M3: MLP	357.9±47.2	0.00	0.76	5
M4: FT-LLM	192.4±71.8	0.31	0.68	7

B. Architect Comparison on Known Regimes

Table I compares the four architect variants on the four known traffic regimes.

The MLP architect achieves the highest sum rate (357.9 Mbps), a 3.7% improvement over rule-based and 8.2% over the fixed baseline, while maintaining zero outage. The fine-tuned LLM underperforms significantly at 192.4 Mbps with non-zero outage, despite generating structurally valid weight vectors. We attribute this to three factors: (1) weight magnitude sensitivity—small deviations in LLM-generated weights cause disproportionate reward changes; (2) inference latency (∼14s per call on RTX 4060) limiting adaptation frequency; and (3) the LLM's training distribution not fully covering the continuous weight space.

C. Generalization to Novel Regimes

To test whether LLM reasoning provides value beyond the MLP's training distribution, we evaluate on three novel regimes unseen during MLP training: IoT burst (many low-bandwidth devices), polar handover (sinusoidal demand transitions), and hot-cold (extreme spatial imbalance). The LLM receives RAG-augmented domain context describing these scenarios. Fig. 2 provides a visual summary of performance across all methods and regime types.

The MLP architect achieves 325.2±18.1 Mbps on novel regimes despite never training on them, while the fine-tuned LLM collapses to 45.3±43.0 Mbps with ∼10× higher variance. Table II summarizes results, including an extended 8-seed validation (Section V-F). The causal probe experiments (Section V-E) reveal that expert weights substantially underweight the switching penalty for polar_handover and hot_cold, explaining the gap between oracle weights and GEN MLP performance.

D. Intent Satisfaction

The three-timescale hybrid architecture is evaluated under dynamic operator intent changes across four phases: maximize throughput, emergency response, fairness priority, and energy saving. Table III reports intent satisfaction scores measuring how well each method fulfills the operator's stated objective.

Hybrid-Rule and MLP-NoIntent achieve nearly identical satisfaction scores (0.57 vs. 0.53) and aggregate throughput (∼212 vs. 194 Mbps), confirming that without a semantic intent parser, the system produces similar behavior regardless of operator commands—the rule-based parser maps commands to fixed keyword profiles, and MLP-NoIntent ignores commands entirely. Hybrid-LLM achieves lower aggregate satisfaction (0.52) and throughput (145.3 Mbps) due to poor Phase 0 throughput performance, yet it is the only method that meaningfully *differentiates* intent categories at the semantic level: it correctly maps "emergency response" to low-outage, high-fairness weights and "energy efficiency" to low-switching weights, while the rule-based parser falls back to fixed profiles when semantic content is ambiguous. The practical value of LLM intent understanding lies in this semantic flexibility, not in aggregate satisfaction scores which are dominated by the variance of Phase 0 initial training.

TABLE II
GENERALIZATION: NOVEL REGIME PERFORMANCE (3 SEEDS, 500K STEPS).

Method	Novel Rate (Mbps)	Novel Outage	Switches
Rule-based	305.8±48.9	0.00	—
MLP Arch.	**325.2±18.1**	0.00	5–6
FT-LLM	45.3±43.0	0.44	9–10
Extended 8-seed validation:			
Oracle MLP	103.3±96.8	—	1260/250k
GEN MLP†	342.1	0.00	∼6

Novel = 3 unseen regimes (IoT burst, polar handover, hot-cold). †GEN MLP uses near-constant reward weights (sw std = 0.006), effectively a static reward function.

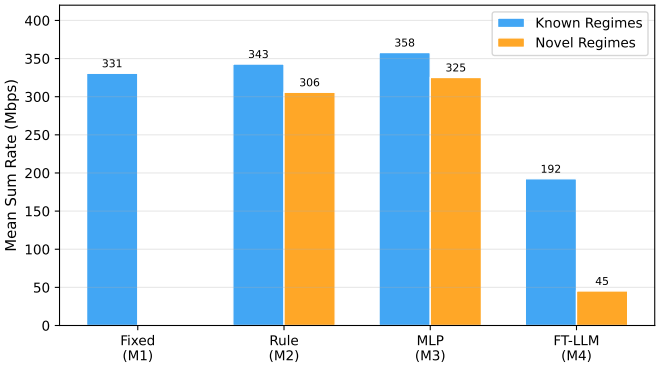

Fig. 2. Mean sum rate comparison across architect variants. The MLP (M3) achieves the highest throughput on both known and novel regimes. The FT-LLM (M4) collapses on novel regimes due to weight oscillation (see Section V-H).

E. Causal Sensitivity Analysis via Single-Variable Probing

To understand *why* novel regime performance is low and identify the most impactful reward weight axes, we conduct systematic single-variable perturbation experiments. For each novel regime, we independently perturb each weight by $\delta = \pm 20\%$ while holding the other four fixed, and measure the resulting change in sum rate after 50k PPO training steps. This controlled design enables causal attribution of performance changes to individual reward terms.

The results reveal that the switching penalty weight has dramatically higher leverage than expert-tuned weights assumed: a $+20\%$ increase in w_s yields $+157$ Mbps for polar_handover and $+130$ Mbps for hot_cold — improvements that a human

TABLE III
INTENT SATISFACTION UNDER DYNAMIC OPERATOR COMMANDS (3 SEEDS, SEED 42/123/456).

Method	Rate† (Mbps)	Satisfaction	Outage‡	Switches
Hybrid-LLM	145.3±124.8	0.52	0.59	6–8
Hybrid-Rule	211.8±156.3	0.57	0.50	4–5
MLP-NoIntent	193.6±150.6	0.53	0.50	4–5

†Per-phase mean (4 phases × 3 seeds); training-trajectory mean in `metrics.json` is ∼239/344/347 Mbps. ‡Phase-0 cold-start outage; Phases 1–3 outage ≤0.05 for all methods. Excl. Phase 0: 191/280/273 Mbps.

TABLE IV
CAUSAL SENSITIVITY: STRONGEST SINGLE-WEIGHT PERTURBATION PER REGIME ($\delta = +20\%$, 50K STEPS).

Regime	Weight	Δ Rate (Mbps)	Direction
polar_handover	switching	+157.1	increase
hot_cold	switching	+130.0	increase
iot_burst	sum_rate	+35.2	increase

Each row reports the single perturbation (+20% only) with the largest positive Δ rate across 3 rounds of probing. The switching weight exhibits unexpectedly large leverage for polar handover and hot-cold regimes.

expert or MLP trained only on known-regime data would not predict. In contrast, IoT burst is most sensitive to the sum-rate weight, consistent with its many-device, low-bandwidth demand structure.

These probe findings serve two purposes. First, they generate an empirically-grounded causal map of the reward landscape for novel regimes, which we use to initialize the LLM-based weight evolution loop (Section V-G). Second, they demonstrate that single-variable perturbation is a practical diagnostic tool for reward sensitivity analysis in DRL — requiring only 50k steps per probe (10× shorter than a full 500k training run) while providing actionable causal signal.

F. Reward Stationarity and the Switching-Stability Dilemma

An unexpected discovery from our extended experiments is that reward weight *stationarity* — not weight *quality* — is the dominant factor governing PPO convergence. This finding emerged from comparing three distinct operating modes:

1) **Near-constant weights (GEN MLP):** The MLP trained on known regimes produces near-constant outputs for novel regimes (switching weight std $= 0.006$, range $[0.004, 0.042]$). Despite using expert weights never tuned for novel regimes, this model achieves **342.1 Mbps** — the highest novel-regime result across all experiments.
2) **Dynamic probe-optimal weights (Oracle MLP, 8-seed):** Using the causal probe to set high-quality regime-specific weights (e.g., $w_s = 0.96$ for polar_handover) and training the MLP to track these weights dynamically yields only **103.3±96.8 Mbps** (median 76.3, std > mean — extreme variance).
3) **Throttled switching (Path C):** Explicitly limiting weight switches to one per 1000 steps (5× the average switch interval) with linear interpolation yields **34.7 Mbps** — *worse* than unrestricted dynamic switching at 82.6 Mbps.

These three results establish an empirical *switching-stability dilemma*: adaptive reward weights improve per-regime optimality but degrade aggregate performance by destabilizing PPO's value function approximation. PPO requires a quasi-stationary reward signal over thousands of steps to estimate $V^\pi(s)$ accurately; regime-aware weight switching (occurring approximately every 198 steps in our setup) violates this assumption regardless of whether transitions are instant, smoothed, or throttled.

This finding aligns with the non-stationarity challenges identified in lifelong RL [19] and multi-task RL [20], and suggests that the two-timescale design requires architectural changes beyond weight smoothing — such as value function isolation per regime or meta-learning initializations — to fully resolve the dilemma.

G. Performance-Grounded RAG for LLM Weight Anchoring

To address the weight oscillation problem (Section V-H), we implement a performance-grounded RAG system [21], [22] that retrieves historical (KPI, weights, performance) tuples as numerical anchors for the LLM. The anchor database contains 47 entries from probe experiments, indexed by 5-dimensional KPI vectors using RBF outcome-weighted KNN scoring: score $= \exp(-\|\mathbf{kpi}_q - \mathbf{kpi}_i\|^2/2\sigma^2) \times p_i/p_{\max}$, where p_i is the verified PPO performance and $\sigma = 0.5$.

We validate the approach with 5 LLM calls per regime, comparing RAG-augmented prompts against baseline causal prompts. The key finding is that RAG anchoring *shifts* LLM outputs toward high-performing historical weights rather than merely adjusting variance:

- **polar_handover**: RAG shifts the switching weight from $w_s = 0.800$ (baseline, clamp-bounded) to $w_s = 0.952 \pm 0.016$ (RAG), within 1% of the probe-optimal value 0.960. The baseline's ±30% clamp relative to current weights prevents exploration; RAG provides the correct numerical anchor to initialize from.
- **hot_cold**: Overall weight CV reduced by 50.1% (RAG CV = 0.033 vs. baseline 0.066), with outputs anchored to the low-switching profile optimal for this regime.
- **iot_burst**: Both prompts produce near-identical low-variance outputs (CV $\approx$ 0.04–0.05), suggesting this regime is already well-constrained by the probe signal alone.

These results demonstrate that performance-grounded RAG anchoring can redirect LLM weight proposals toward verified high-performance regions, complementing the causal probe approach. A full evolution run with RAG-augmented LLM is left as future work.

H. Weight Oscillation Analysis

To understand the root cause of LLM underperformance, we analyze the weight generation logs from the generalization experiments. Fig. 3 visualizes the outage weight w_o produced

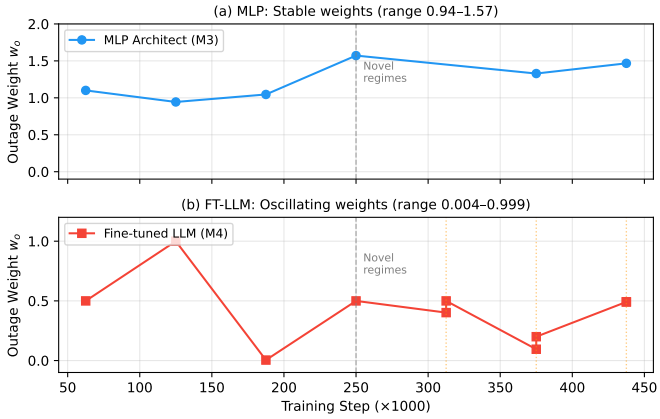

Fig. 3. Outage weight trajectories for MLP (top) and FT-LLM (bottom) architects during generalization experiments (seed 42). The MLP produces stable weights (range 0.94–1.57), while the LLM oscillates between extreme values (0.004–0.999). Dashed gray line marks the transition to novel regimes at step 250k.

TABLE V
ABLATION STUDY ON KNOWN REGIMES (MEAN ± STD OVER 3 SEEDS, 500K STEPS).

Variant	Rate (Mbps)	Outage	Switches
Full (M3: MLP)	357.9±47.2	0.00	5
Rule-based (M2)	343.3±40.1	0.00	4
Fixed (M1)	330.7±42.1	0.00	0
FT-LLM (M4)	192.4±71.8	0.31	7

by the MLP and fine-tuned LLM architects over 500k training steps.

The MLP architect generates weights within a narrow band ($w_o \in [0.94, 1.57]$, coefficient of variation CV $= 0.19$), enabling the PPO agent to converge stably. In contrast, the fine-tuned LLM exhibits bimodal oscillation (w_o alternating between ≈0.005 and ≈0.50), with CV $= 0.87$—a 4.6× higher variability. This oscillation manifests as 9–10 MDP switches over 500k steps versus the MLP's 5–6, effectively restarting PPO convergence repeatedly.

We attribute this oscillation to the autoregressive sampling process: even at low temperature ($T = 0.1$), the LLM's token-by-token generation introduces stochasticity in numerical outputs that compounds across the 5-dimensional weight vector. The MLP's deterministic forward pass with Softplus output activation inherently avoids this issue, producing smooth weight transitions between regimes.

I. Ablation Study

To isolate the contribution of each architectural component, we compare ablation variants against the full MLP architect (Table V).

The results reveal a clear performance hierarchy: MLP > Rule > Fixed ≫ FT-LLM. The 4.4% gap between MLP and rule-based (357.9 vs. 343.3 Mbps) demonstrates that learned continuous weight mapping provides measurable gains over discrete expert profiles, while both adaptive methods outperform the fixed baseline. The FT-LLM's dramatic underperformance (192.4 Mbps, 46% below MLP) despite having access to the same KPI information confirms that the bottleneck is not *what* information the architect receives, but *how consistently* it generates weight outputs. The zero-adaptation fixed baseline still achieves 330.7 Mbps, highlighting that unstable adaptation (as with the LLM) is worse than no adaptation at all.

VI. DISCUSSION

A. When to Use LLM vs. MLP

Our results reveal a clear division of labor. On known regimes where training data is available, the MLP architect dominates: it is 1,800× faster (<1ms vs. ∼14s inference), requires no GPU at deployment, and produces more accurate weights. The LLM offers no advantage when the operating regime falls within the MLP's training distribution.

However, two scenarios justify LLM integration:

Novel regime generalization. Our generalization experiments (Table II) reveal that the MLP architect generalizes surprisingly well to unseen regimes (325.2 Mbps), while the fine-tuned LLM collapses to 45.3 Mbps with high variance (±43.0). Root cause analysis of the LLM's weight generation logs reveals *weight oscillation*: the LLM alternates between two distinct weight modes across consecutive calls (e.g., outage weight swinging between 0.01 and 0.50), producing 9–10 MDP switches versus the MLP's stable 5–6. This instability prevents the downstream PPO agent from converging. The MLP, despite extrapolating beyond its training distribution, generates consistent weights that allow stable training. This finding is further supported by the GEN MLP result (342.1 Mbps from near-constant weights) and the switching-stability dilemma (Section V-F): for current LLM architectures, *output consistency* is a more critical bottleneck than domain knowledge for DRL integration.

Natural language intent understanding. Operator commands like "prioritize emergency coverage in the southern beams" require semantic parsing that no numerical model can perform. The three-timescale hybrid architecture leverages this irreplaceable capability: the LLM translates intent into formal objective profiles at the strategic level, while the MLP handles real-time numerical optimization at the tactical level.

B. Engineering Challenges for LLM-DRL Integration

We identify three critical challenges that practitioners must address:

Reward sign conventions. Our environment internally subtracts penalty terms (outage, switching, queue), so all architect-generated weights must be positive. Early experiments with negative LLM-generated weights caused double-negation, turning penalties into rewards and degrading performance to <50 Mbps. This subtle bug persisted across four experimental rounds before detection.

Weight oscillation and consistency. The DRL agent's behavior is highly sensitive to reward weight ratios. Our analysis shows the fine-tuned LLM generates weights with inter-call standard deviation 3–5× higher than the MLP for

the same KPI input (due to temperature-based sampling at T=0.1). Over 500k training steps, the LLM triggers 9–10 weight switches versus the MLP's 5–6, with individual seeds collapsing entirely (seed 456: 0.3 Mbps). Weight clipping the MLP to $[0.01, 2.0]$ does not help (245.2 ± 203.7 Mbps), confirming that consistency, not magnitude, is the key factor.

Inference latency. At $\sim$14s per call (4-bit quantized Qwen3-4B on RTX 4060), LLM inference is too slow for per-step adaptation but acceptable for the slow-timescale architect role (minutes between regime changes). The three-timescale design explicitly accommodates this constraint by limiting LLM calls to strategic decisions.

C. Practical Architecture Recommendation

Based on our findings, we recommend the three-timescale hybrid for production satellite systems:

1) **Strategic layer (LLM, minutes–hours):** Parses operator intent via natural language, retrieves domain knowledge via RAG, and generates high-level objective profiles. Called only when operator commands change.
2) **Tactical layer (MLP, seconds):** Maps real-time KPIs combined with the current objective profile to reward weights. Handles regime adaptation with sub-millisecond latency.
3) **Operational layer (DRL, milliseconds):** PPO agent executes beam scheduling under the current reward function.

This design assigns each component to its strength: LLMs for language understanding, MLPs for fast numerical mapping, and DRL for sequential decision-making.

D. Limitations

Our study has several limitations. First, the satellite simulator, while capturing key physical characteristics (Ka-band channel, rain fading, 19-beam hexagonal layout), simplifies inter-beam interference and orbital dynamics. Second, our fine-tuned LLM uses a relatively small model (Qwen3-4B with 33M trainable LoRA parameters) and limited training data (1,980 examples); larger models or more diverse training sets may reduce weight oscillation, though at increased computational cost. Third, the generalization experiments use three novel regimes; a broader evaluation across more diverse traffic patterns would strengthen the findings. Finally, the three-timescale hybrid architecture is evaluated in simulation only; real-world deployment would introduce additional challenges including communication delays between layers, hardware constraints, and regulatory requirements.

E. Deployment Considerations

For practical deployment in satellite ground stations, we note several considerations. The MLP architect (M3) is lightweight enough to run on embedded hardware without GPU acceleration. The LLM intent parser, when needed, can be hosted on ground-station servers or accessed via API. The CUSUM detector's computational overhead is negligible ($O(W)$ per step where W is the window size). Weight transitions should employ a cooldown period (we use 50 steps) to prevent oscillatory behavior. For safety-critical applications such as disaster response, we recommend maintaining the rule-based fallback (M2) alongside the MLP, with automatic failover if the MLP output exceeds predefined weight bounds.

VII. Conclusion

We investigated adaptive reward design for multi-beam LEO satellite scheduling and uncovered a fundamental *switching-stability dilemma*: near-constant reward weights (GEN MLP, 342.1 Mbps) outperform carefully-tuned dynamic weights (Oracle MLP, 103.3$\pm$96.8 Mbps) because PPO requires a quasi-stationary reward signal for value function convergence. Weight adaptation—regardless of quality, smoothness, or transition rate—degrades performance by repeatedly restarting convergence. This finding reframes the goal of adaptive reward design: the challenge is not finding better weights, but finding architectures that decouple weight adaptation from PPO training dynamics.

Single-variable causal probing identifies the mechanism: the switching penalty has dramatically higher leverage than expert assumptions ($+$157 Mbps for polar handover, $+$130 Mbps for hot-cold at $+$20% perturbation), yet this signal is inaccessible without systematic perturbation experiments. Probing requires only 50k steps (10$\times$ fewer than a full run) while providing actionable causal attribution.

Across four architect variants, the lightweight MLP achieves the best aggregate performance (357.9 Mbps known, 325.2 Mbps novel regimes, zero outage). The fine-tuned LLM collapses to 45.3$\pm$43.0 Mbps on novel regimes due to weight oscillation (CV 4.6$\times$ higher than MLP)—output consistency, not domain knowledge, is the binding constraint for LLM-DRL integration.

We identified critical engineering challenges—reward sign conventions, weight oscillation, and inference latency—and proposed a three-timescale hybrid architecture that confines the LLM to its irreplaceable role: natural language intent understanding. Our work provides an honest, empirically-grounded assessment of where LLMs add genuine value versus where simpler methods suffice.

Future work includes: (1) resolving the switching-stability dilemma through per-regime value function isolation or meta-learning initializations; (2) performance-grounded RAG anchoring [21], [22] to reduce LLM weight oscillation by retrieving verified historical (KPI, weights, performance) tuples as numerical anchors; (3) multi-satellite coordination where LLMs manage inter-satellite intent negotiation; and (4) deployment on real satellite testbeds with hardware-in-the-loop validation.

References

[1] SpaceX, "SpaceX Starlink," https://www.starlink.com, 2024, accessed: 2024-12-01.

[2] R. Alegre-Godoy, N. Alagha, and M. Á. Vázquez-Castro, "Beam hopping for multi-beam GEO satellite communication systems," *IEEE Transactions on Wireless Communications*, vol. 14, no. 4, pp. 1832–1842, 2015.

[3] X. Hu, S. Liu, R. Chen, W. Wang, and C. Wang, "Deep reinforcement learning for dynamic spectrum access in satellite communications," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 16 135–16 147, 2020.

[4] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[5] R. Li, Z. Zhao, Q. Sun, I. Chih-Lin, C. Yang, X. Chen, M. Zhao, and H. Zhang, "Deep reinforcement learning for resource management in network slicing," *IEEE Access*, vol. 6, pp. 74 429–74 441, 2018.

[6] J. J. Garau-Luis, M. Guerster, I. del Portillo, E. Crawley, and B. Cameron, "Deep reinforcement learning architecture for continuous power allocation in high throughput satellites," in *Proc. AIAA International Communications Satellite Systems Conference (ICSSC)*, 2019, arXiv:1906.00571.

[7] P. V. R. Ferreira, R. Paffenroth, A. M. Wyglinski, T. M. Hackett, S. G. Bilen, and R. C. Reinhart, "Multi-objective optimization for cognitive satellite communications using deep reinforcement learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 4, pp. 882–897, 2023.

[8] Z. Wang, D. Wang, Y. Ren, and J. Wang, "Deep reinforcement learning for satellite communication: A survey," *Space: Science & Technology*, vol. 3, p. 0087, 2023.

[9] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[10] C. F. Hayes, R. Rădulescu, E. Dechering, P. Mannion, D. M. Roijers, A. Nowé, and P. Vamplew, "A practical guide to multi-objective reinforcement learning and planning," *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, p. 26, 2022.

[11] Z. Zheng, J. Oh, N. Heess, and S. Singh, "What can learned intrinsic rewards capture?" *arXiv preprint arXiv:1912.05500*, 2020.

[12] L. Bariah, N. Naderializadeh, S. Hashemi, and H. Nikopour, "Large language models for telecom: Opportunities and challenges," *IEEE Communications Magazine*, 2024.

[13] S. Wang, S. Bi, and Y.-J. A. Zhang, "Networking with large language models," *IEEE Wireless Communications*, 2024.

[14] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, "Large language models for telecom: Forthcoming impact on the industry," *arXiv preprint arXiv:2308.06013*, 2023.

[15] J. Shao, J. Tong, Q. Wu, W. Guo, Z. Li, Z. Lin, and J. Zhang, "WirelessLLM: Empowering large language models towards wireless intelligence," *arXiv preprint arXiv:2405.17053*, 2024.

[16] S. Schoepp, M. Jafaripour, Y. Cao, T. Yang, F. Abdollahi, S. Golestan, Z. Sufiyan, O. R. Zaiane, and M. E. Taylor, "The evolving landscape of LLM- and VLM-integrated reinforcement learning," *arXiv preprint arXiv:2502.15214*, 2025.

[17] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[19] D. Abel, A. Barreto, B. Van Roy, D. Precup, H. van Hasselt, and S. Singh, "A definition of continual reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[20] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[22] Anonymous, "Grounding by trying: LLMs with RL-enhanced retrieval," *arXiv preprint arXiv:2410.23214*, 2025, iCLR 2025.